Medical Record Semantic Retrieval Internship Task

> **Assignment:** Build a Python-based semantic retrieval prototype for medical records using embeddings. The goal is retrieval quality evaluation, not diagnosis, summarization, or answer generation.

Duration: 3 working days
Project type: Python experimentation and retrieval prototype
Core techniques: Text embeddings, cosine similarity, dot product, top-k retrieval, PCA, UMAP
Minimum model count: 3 embedding models
Minimum dimensionality settings: 3
Dataset: JSON records supplied by mentor
Final output: Runnable code, results, visualizations, and report

Task Overview

Build a semantic retrieval system that finds existing medical records relevant to a free-text query. The system must encode both records and queries into vectors, rank records using similarity measures, and return the top-k most similar matches with **record ID**, **score**, and **original text**.

> **Out of scope:** No LLMs, no chatbot behavior, no generated diagnoses, no medical advice, and no retrieval-augmented answer generation. This project retrieves and displays existing records only.

Learning Objectives

- Explain how text is represented as a high-dimensional vector.
- Compare retrieval behavior across at least three embedding models.
- Understand cosine similarity versus dot product, including normalization effects.
- Evaluate how embedding dimensionality affects quality, memory, and latency.
- Use PCA and UMAP for visualization without confusing 2D plots with retrieval space.
- Implement configurable top-k retrieval and evaluate it with relevance labels.
- Communicate findings through reproducible experiments, tables, plots, and limitations.

Scope and Constraints

- No LLM answer generation, summarization, diagnosis, triage, or treatment suggestions.
- Use only the mentor-supplied or mentor-approved JSON dataset.
- Synthetic data is preferred.
- Do not upload identifiable patient information to external APIs or services.
- All model comparisons must use the **same** corpus, query set, and relevance labels.
- Do not interpret a high similarity score as clinical correctness.

Dataset Requirements

The mentor will provide a JSON file containing medical-record text and metadata. Field names may vary, but record IDs must be **stable** and **unique**.

Recommended record schema

```json
[
  {
    "record_id": "MR0001",
    "note_text": "Patient reports persistent cough and shortness of breath.",
    "category": "respiratory",
    "source": "synthetic_clinical_notes"
  },
  {
    "record_id": "MR0002",
    "note_text": "Patient describes dyspnea with a dry cough.",
    "category": "respiratory",
    "source": "synthetic_clinical_notes"
  },
  {
    "record_id": "MR0003",
    "note_text": "Patient reports intermittent knee pain after exercise.",
    "category": "musculoskeletal",
    "source": "synthetic_clinical_notes"
  }
]
```

Field	Type	Description
record_id	String	Unique stable identifier for each record.
note_text	String	Text to embed and retrieve.
category	String	Recommended label for analysis and visualization.
source	String	Optional dataset or source label.

Evaluation queries should be stored separately and include known relevant record IDs.

Recommended query schema

```json
[
  {
    "query_id": "Q001",
    "query_text": "difficulty breathing and persistent cough",
    "relevant_record_ids": ["MR0001", "MR0002"]
  },
  {
    "query_id": "Q002",
    "query_text": "knee discomfort after working out",
    "relevant_record_ids": ["MR0003"]
```

```
  }
]
```

If the supplied dataset does not include relevance labels, create a small labeled query set with the mentor before running evaluation. Do not infer labels from model output.

Required Methodology

1. Inspect and prepare the dataset

- Load the JSON files and validate required fields.
- Check unique IDs, missing text, duplicates, and text lengths.
- Keep a record-to-embedding mapping for full traceability.
- Document preprocessing choices.
- Avoid removing clinically meaningful negations, numbers, units, or abbreviations without testing.
- Use one fixed query set across all experiments.

2. Evaluate at least 3 embedding models

Select at least three models appropriate for sentence or passage retrieval. Record model name, version, embedding dimension, pooling or encoding approach, normalization behavior, runtime, and hardware requirements.

Model option	Why consider it	Notes
sentence-transformers/all-MiniLM-L6-v2	Lightweight general-purpose sentence embedding baseline.	Useful speed baseline; not trained specifically for clinical text.
sentence-transformers/all-mpnet-base-v2	General-purpose embeddings with a different quality/speed trade-off.	Compare quality and runtime against MiniLM.
Mentor-approved biomedical or clinical embedding model	Tests whether domain-focused training improves retrieval.	Document exact model ID, license, and task suitability.

3. Experiment with at least 3 dimensionality settings

- Record each model's native embedding dimension.
- Test at least three settings, such as native dimension, medium reduction, and smaller reduction.
- Use a documented dimensionality reduction method such as PCA for reduced settings.
- Fit PCA on corpus embeddings only, then transform query embeddings with the same fitted reducer.
- Evaluate retrieval quality, latency, and storage at every setting.

- Clearly distinguish native dimensions from post-processed reduced vectors.

> **Example settings:** native dimension, 128 dimensions, and 64 dimensions, when feasible. If a model's native dimension is already 128 or lower, choose three feasible settings and document why.

4. Compare cosine similarity and dot product

Implement and test both similarity measures using the same corpus and query vectors.

$$\text{cosine}(A, B) = (A \cdot B) / (||A|| \times ||B||)$$
$$\text{dot_product}(A, B) = A \cdot B$$

- Run cosine and dot product on normalized vectors and verify rankings are approximately identical.
- Run dot product on unnormalized vectors and compare ranking changes.
- Document whether normalization comes from the model, your code, or neither.
- Do not treat scores from different models as calibrated probabilities.

5. Implement top-k retrieval and analyze k

- The retrieval function must accept a query and configurable k.
- Return the top k highest-ranked **distinct** records.
- Include record ID, original text, rank, and similarity score.
- Test at least k = 1, 3, 5, and 10 where corpus size permits.
- Handle edge cases consistently: k <= 0, k larger than corpus size, empty queries, blank notes, and ties.
- Precompute corpus embeddings and reuse them across queries.
- Measure Recall@k, Precision@k, MRR, latency, and returned-result count.

6. Visualize with PCA and UMAP

- Create one 2D PCA plot and one 2D UMAP plot from original high-dimensional embeddings.
- Color points by category if labels exist.
- Add hover labels or provide a supporting ID table if using interactive plots.
- Record UMAP parameters such as n_neighbors, min_dist, metric, and random seed.
- Use visualization for exploration only, not for actual retrieval.

7. Evaluate and compare

Metric	Definition / use
Recall@k	Relevant records in top k divided by the total number of known relevant records for that query, averaged across queries.

Precision@k	Relevant records in top k divided by k. If fewer than k records exist, document the convention used.
MRR	Mean reciprocal rank of the first relevant result.
Query latency	Time to encode a query and retrieve results; report whether model loading is excluded and provide median latency.
Embedding storage	Approximate bytes required to store corpus vectors for each model and dimension setting.

Report aggregate metrics on the same query set and include per-query examples. Keep preprocessing, k values, dataset, labels, and evaluation code consistent across comparisons.

Three-Day Work Plan

Day	Work to complete	Expected checkpoint
Day 1	Inspect JSON data; define query and relevance set; implement encoding with 3 models; record native dimensions; implement cosine and dot product.	Validated data, reproducible embeddings, initial retrieval baseline.
Day 2	Run at least 3 dimensionality settings; test normalized and unnormalized vectors; implement configurable top-k; evaluate metrics; generate PCA and UMAP plots.	Saved experiment outputs, visualizations, and metric comparisons.
Day 3	Finish comparisons; analyze failures; prepare Streamlit or CLI demo; write README and final report; present results.	Runnable project, report, plots, results tables, and demo.

Required Deliverables

- [ ] Source code with clear README, reproducible setup instructions and basic architecture diagram
- [ ] Dataset loader and validation script for supplied JSON files

- [ ] Retrieval implementation with model, similarity metric, dimensionality, and top-k configuration
- [ ] Results comparing at least 3 embedding models
- [ ] Results for at least 3 dimensionality settings
- [ ] Cosine-versus-dot-product comparison, including normalized and unnormalized behavior
- [ ] Top-k analysis for k = 1, 3, 5, and 10 where possible
- [ ] PCA and UMAP visualizations with parameters documented
- [ ] Evaluation table with Recall@k, Precision@k, MRR, latency, and storage size
- [ ] At least 5 analyzed retrieval examples, including successes and failure cases
- [ ] Final report and a 5–10 minute demo or presentation

Acceptance Criteria

- At least 3 embedding models are run and compared on the same query set.
- At least 3 embedding dimensionality settings are evaluated and clearly described.
- Both cosine similarity and dot product are implemented and normalization behavior is explained.
- Top-k is configurable, edge cases are handled, and k sensitivity is measured.
- PCA and UMAP plots are included with settings documented.
- Evaluation uses known relevance labels and includes Recall@k, Precision@k, and MRR.
- The code runs from documented instructions and results are reproducible.
- The report includes limitations and does not present retrieval as diagnosis or decision support.

Evaluation Rubric

Area	Weight	What good work looks like
Embedding model experiments	20%	Three or more models compared fairly; model details and trade-offs documented.
Dimensionality experiments	15%	Three or more dimensions tested; effects on retrieval, latency, and storage explained.
Similarity and top-k handling	20%	Correct cosine and dot-product implementation, normalization analysis, and robust k behavior.
Evaluation and visualization	20%	Correct metrics, consistent test set, meaningful PCA/UMAP plots, and failure analysis.

Code quality and reproducibility	15%	Clear structure, validation, setup instructions, and reusable components.
Documentation and demo	10%	Clear report, evidence-backed conclusions, and concise demonstration.

Final Questions for the Intern

- Which embedding model performed best on the labeled queries, and what trade-offs did it make?
- Did reducing embedding dimensionality affect retrieval quality, storage, and latency?
- When did cosine similarity and dot product produce identical rankings? When did they differ, and why?
- How did changing k affect recall and precision?
- Did PCA and UMAP show clear clusters, and did those clusters align with retrieval quality?
- Which query types failed, such as abbreviations, negation, rare terms, or similar symptoms with different meanings?
- What would you change in a next iteration, and what evidence supports that change?

Safety and Limitations

⚠ This is an educational information-retrieval project, not a clinical system. Use synthetic or properly approved de-identified data only. Do not include names, dates of birth, addresses, medical record numbers, or other direct identifiers in reports or screenshots.

- Do not send sensitive data to external embedding services unless explicitly approved.
- Similarity scores are not probabilities.
- Retrieved records must not be used to diagnose, triage, or recommend treatment.

Suggested Project Structure

Component	Purpose	Notes
data_loader.py/ ipynb	Load and validate JSON records and queries	Check schema, nulls, duplicates, and stable IDs
embed.py/ ipynb	Generate corpus and query embeddings	Support model selection and optional normalization

reduce_dim.py/ ipynb	Apply PCA-based dimensionality reduction	Fit on corpus only, transform queries with same reducer
retrieve.py/ ipynb	Similarity scoring and top-k ranking	Support cosine and dot product
evaluate.py/ ipynb	Compute Recall@k, Precision@k, MRR, latency, storage	Keep evaluation consistent across runs
visualize.py/ ipynb	Create PCA and UMAP plots	Document parameters and seeds
app.py or cli.py/ ipynb	Simple demo interface	Streamlit or CLI is acceptable

Reference Documentation

- SentenceTransformers Documentation — Sentence Transformers documentation
- PCA
- UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction — umap 0.5.8 documentation
- cosine_similarity